

CLOUD ENGINEERING BOOTCAMP · DAY 2

Linux Fundamentals

Navigating, managing, and securing a Linux system — the foundation every cloud engineer builds on.

DURATION

2 – 3 hours

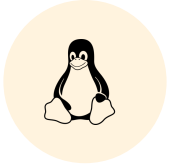
AUDIENCE

Absolute beginners — no IT background required



Learning Objectives

By the end of today, you will be able to:



Explain what Linux is and why cloud engineers depend on it



Navigate the Linux file system using the command line



Create, copy, move, and delete files and directories safely



Understand users, groups, and file permissions



Monitor and manage running processes and services



Connect to a remote Linux server securely using SSH

Today's Agenda

1 Introduction to Linux & Distributions

2 Linux Architecture & File System

3 Navigation & File Management

4 Users, Permissions & Processes

5 SSH & Remote Access

6 Hands-On Lab

7 Quiz & Wrap-Up

Let's Warm Up



“What operating systems have you used?”

Windows · macOS · Chrome OS · Android · Linux · None yet — and that's OK!

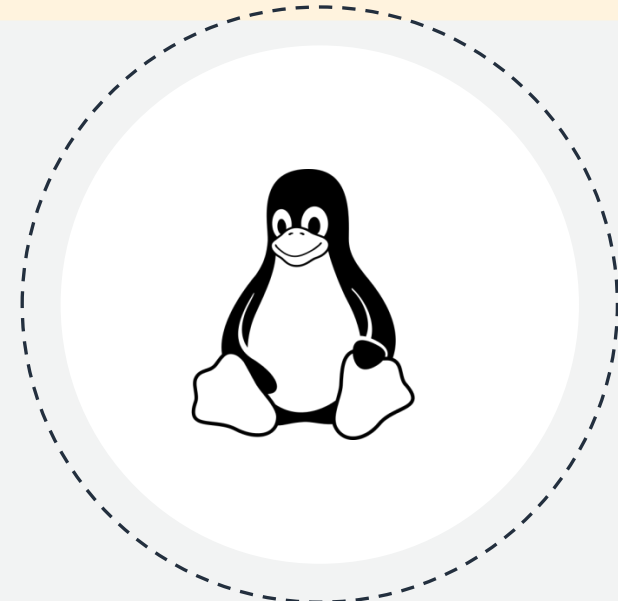
What Is Linux?

OBJECTIVE: Understand what Linux is and why it matters for cloud engineering.

Linux is a free, open-source operating system — like Windows or macOS — that runs on servers, phones, and even the AWS cloud itself.

- Free & open-source
- Highly stable & secure
- Runs most of the Internet

REAL-WORLD ANALOGY: Linux is like the engine of a car — you may never see it, but it's what powers almost everything under the hood.



Tux, the Linux mascot

A Brief History

OBJECTIVE: See how Linux evolved from a hobby project into the backbone of the cloud.



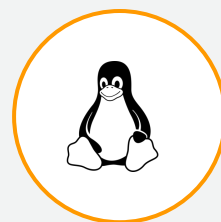
1969

UNIX is created at Bell Labs



1983

The GNU Project begins, aiming for a free UNIX-like OS



1991

Linus Torvalds releases the Linux kernel



2000s+

Linux becomes the standard for servers & the cloud



INSTRUCTOR TIP: Say “Linux” = kernel only. Say “GNU/Linux” = kernel + all the surrounding tools that make it usable.

Why Cloud Engineers Use Linux

Most of the cloud runs on Linux — here's why engineers prefer it.



Free & Open-Source

No licensing costs at scale



Rock-Solid Stability

Servers run for years without reboot



Powerful Automation

Scriptable for repetitive tasks



Strong Security

Granular permission controls

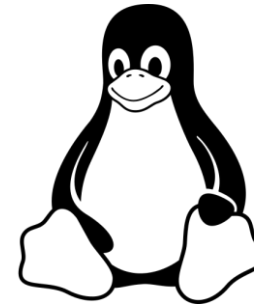
Over 90% of public cloud workloads run on Linux.

What Is a Distribution?

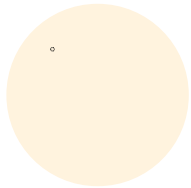
OBJECTIVE: Understand that “Linux” is a family of operating systems, not just one.

A “distribution” (or “distro”) is the Linux kernel packaged together with tools, software, and a package manager — ready to install and use.

REAL-WORLD ANALOGY: Think of the kernel as a car engine, and each distro as a different car built around that same engine — a sedan, a truck, a sports car.



Popular Linux Distributions



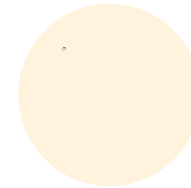
Ubuntu

Beginner-friendly, huge community



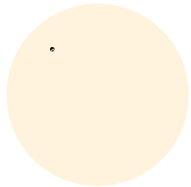
Amazon Linux

AWS's own, optimized for EC2



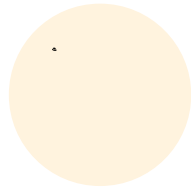
Debian

Stable, the base for Ubuntu



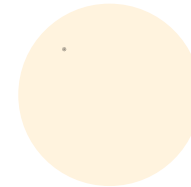
Rocky Linux

Enterprise-grade, free



Red Hat (RHEL)

Enterprise standard, paid support



CentOS

Community rebuild of RHEL

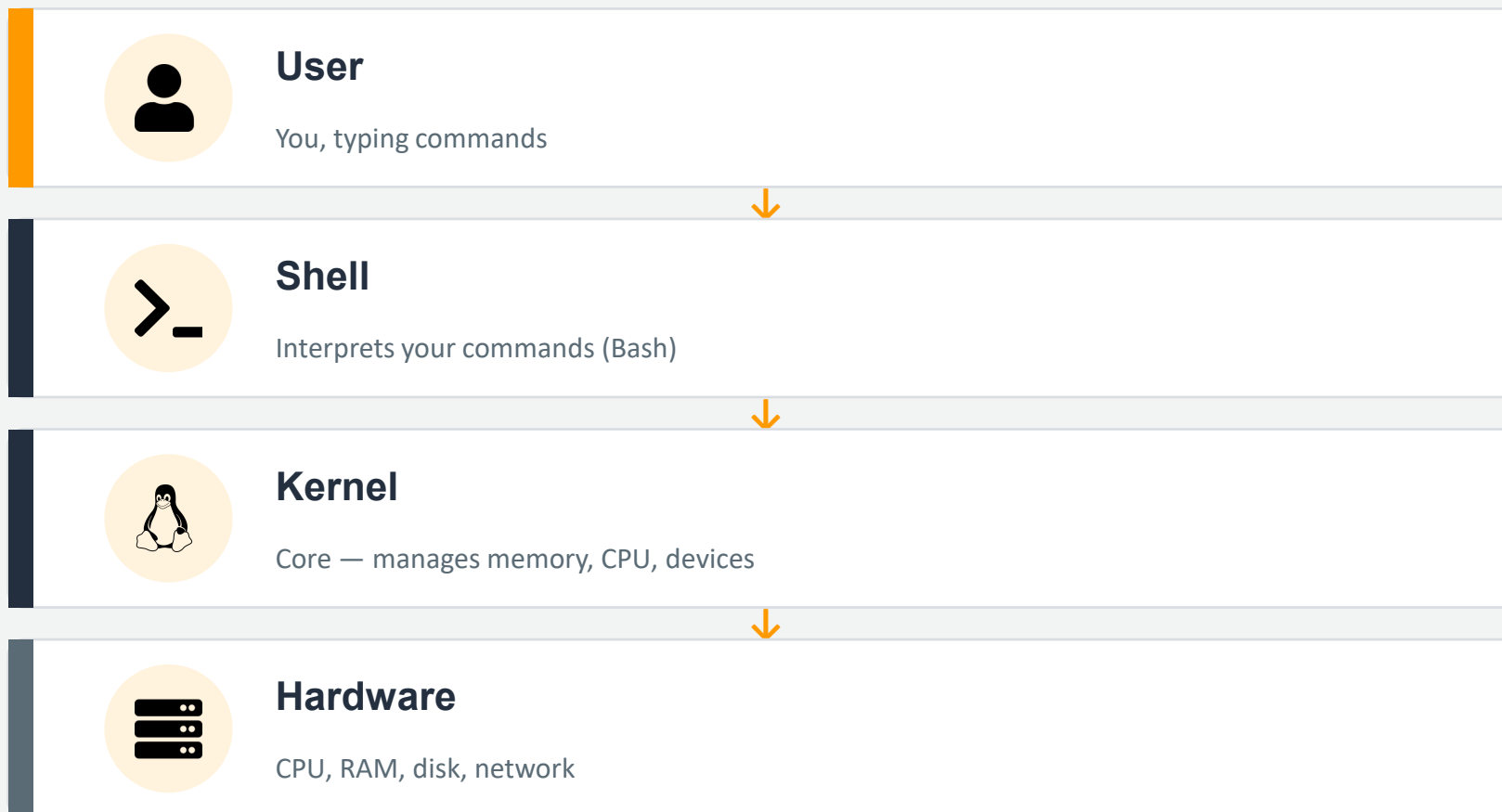
Windows vs. Linux

OBJECTIVE: Compare Linux to the OS most students already know.

CATEGORY	WINDOWS	LINUX
Filesystem	C:\, D:\ drive letters	Single tree, starts at /
Security	Fewer built-in controls	Granular permissions by default
Cost	Paid licenses	Free & open-source
CLI	PowerShell / CMD (optional)	Bash — primary interface
GUI	Always on by default	Optional — often disabled on servers
Cloud Usage	Common for enterprise apps	Dominant for servers & cloud

How Linux Is Structured

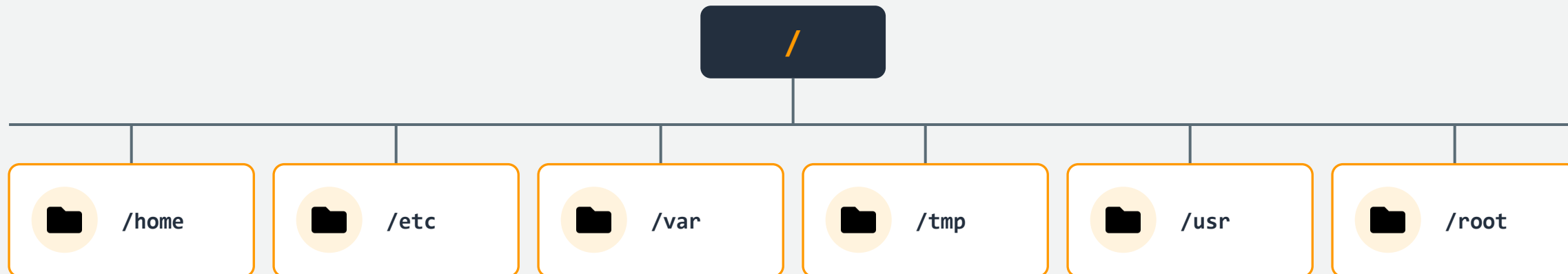
OBJECTIVE: See how your commands travel from you down to the physical hardware.



Everything Starts at /

OBJECTIVE: Understand the tree structure that organizes every file on a Linux system.

REAL-WORLD ANALOGY: Think of / as the root of a tree — every folder and file branches out from that single starting point.



What Lives Where



/home

Personal folders for each user



/etc

System-wide configuration files



/var

Variable data — logs, caches, mail



/tmp

Temporary files, cleared on reboot



/usr

Installed user programs & libraries



/root

Home folder for the root (admin) user

Finding Your Way Around

Five core commands to move through the file system with confidence.



pwd — Print Working Directory

OBJECTIVE: Find out exactly where you are in the file system.

```
user@linux-server: ~  
  
$ pwd  
/home/student
```

REAL-WORLD ANALOGY: Like checking the “You Are Here” marker on a mall map before deciding where to walk.

WHY IT MATTERS

Always confirm your location before running commands that create, move, or delete files — especially with `sudo`.



INSTRUCTOR TIP: Make `pwd` a reflex — run it any time you feel unsure of your location.

ls — List Directory Contents

OBJECTIVE: See the files and folders inside your current location.

```
user@linux-server: ~  
  
$ ls  
Documents Downloads Pictures  
  
$ ls -l  
drwxr-xr-x  2 student student 4096 Documents  
  
$ ls -a  
.  ..  .bashrc  Documents  Downloads
```

COMMON OPTIONS

ls -l Long format — permissions, size, date

ls -a Show hidden files (start with .)

ls -lh Long format, human-readable sizes



INSTRUCTOR TIP: `ls -la` (combined) is the single most-used command for beginners — teach it early.

cd — Change Directory

OBJECTIVE: Move between folders using relative and absolute paths.

```
user@linux-server: ~  
  
$ cd Documents  
  (moved into Documents)  
  
$ cd ..  
  (moved up one level)  
  
$ cd /var/log  
  (jumped straight to /var/log)  
  
$ cd ~  
  (returned to your home folder)
```

PATH TYPES

Absolute

Starts at / — e.g. /var/log

Relative

Starts from where you are — e.g. ../log

Shortcuts

. = here .. = up one ~ = home

mkdir & touch — Create Folders and Files

OBJECTIVE: Create new directories and empty files from the command line.

```
user@linux-server: ~  
  
$ mkdir projects  
  (new folder created)  
  
$ touch notes.txt  
  (new empty file created)  
  
$ ls  
notes.txt  projects
```



mkdir

“Make Directory” — creates a new, empty folder.



touch

Creates a new, empty file (or updates its timestamp).



INSTRUCTOR TIP: Folder and file names are case-sensitive and shouldn't contain spaces — use hyphens or underscores instead.

Navigation Recap

pwd

Where am I?

ls

What's here?

cd

Move somewhere else

mkdir

Make a folder

touch

Make a file

Copy, Move & Edit

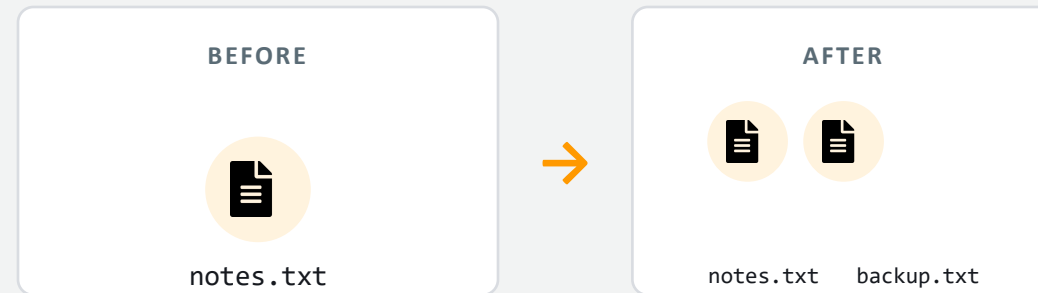
Manage the lifecycle of your files directly from the terminal.



cp — Copy Files

OBJECTIVE: Duplicate a file or folder without removing the original.

```
user@linux-server: ~  
$ cp notes.txt backup.txt
```



INSTRUCTOR TIP: Use `cp -r` when copying an entire folder — the `-r` means “recursive.”

REAL-WORLD ANALOGY: Just like photocopying a document — the original stays right where it was.

mv — Move or Rename Files

OBJECTIVE: Relocate a file, or rename it — mv does both.

```
user@linux-server: ~  
  
$ mv notes.txt Documents/  
  (file moved into Documents)  
  
$ mv backup.txt backup-old.txt  
  (file renamed)
```

mv = move AND rename

Unlike cp, the original disappears from its old location — there's only ever one copy.

REAL-WORLD ANALOGY: Like physically moving a paper file from one folder to another in a filing cabinet — it only exists in one place at a time.

rm — Remove Files (Use With Care)

OBJECTIVE: Delete files and folders permanently — there is no Recycle Bin.

user@linux-server: ~

```
$ rm old-file.txt  
(permanently deleted)
```

```
$ rm -r old-folder/  
(folder and contents deleted)
```



No Undo

rm deletes immediately and permanently — there is no Recycle Bin or Trash to recover from.



INSTRUCTOR TIP: Always run ls first to confirm exactly what you're about to delete — and never run rm -rf / on any system.

cat, nano & echo — View & Edit Content

OBJECTIVE: Read, create, and edit text file content without leaving the terminal.

user@linux-server: ~

```
$ cat notes.txt
```

```
Hello from Linux!
```

```
$ echo "New line" >> notes.txt
```

```
(appends text to the file)
```

```
$ nano notes.txt
```

```
(opens the file in an editor)
```

cat

Display a file's contents

echo

Print text, or write it to a file with >

nano

A simple, beginner-friendly text editor

Who Can Do What?

Linux's permission system decides exactly who can read, write, or run each file.



Owner, Group & Others

OBJECTIVE: Understand the three levels every Linux file permission applies to.



Owner

The user who created the file



Group

A team sharing access rights



Others

Everyone else on the system

REAL-WORLD ANALOGY: Like an office document: you (owner) can edit it, your team (group) can view it, and the rest of the building (others) can't open it at all.

Read, Write & Execute

OBJECTIVE: Decode the rwx letters shown by ls -l into their real meaning.

r

Read

View file contents

w

Write

Modify or delete

x

Execute

Run as a program

-rwxr-xr--

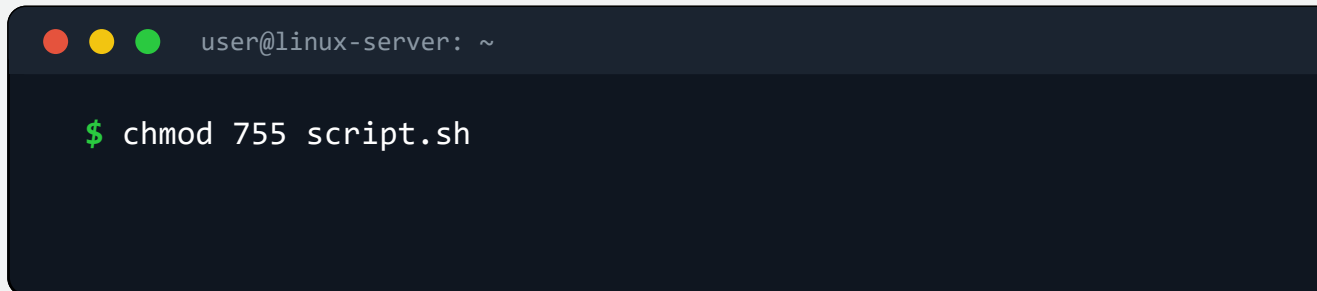
Owner: read, write, execute · Group: read, execute · Others: read only

chmod — Changing Permissions

OBJECTIVE: Use numeric shorthand to set exact permission levels.

r = 4 **w** = 2 **x** = 1

Add the numbers for each of Owner / Group / Others.



```
user@linux-server: ~  
$ chmod 755 script.sh
```

755

rw- r-x r-x

Scripts you run — you edit, others just execute

644

rw- r-- r--

Regular documents — you edit, others read only

777

rw- rw- rw-

Full access for everyone — avoid unless required

Real-World Example

OBJECTIVE: See how permissions protect a real web server.

```
root@web-server: /var/www/html
```

```
$ ls -l /var/www/html
```

```
-rw-r--r--  1 root  root   2048  index.html
-rwxr-xr-x  1 root  www    512   deploy.sh
drwx----- 2 root  root   4096  private/
```

index.html is world-readable so visitors' browsers can load the page — but only root can edit it. deploy.sh is executable for the web group. private/ is fully locked to everyone but root.

Processes, Daemons & Services

OBJECTIVE: Understand the different kinds of programs running on your system.



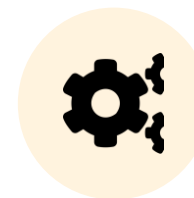
Process

Any running program, e.g. nano



Daemon

A background process with no user interface



Service

A managed daemon (start/stop/restart)

REAL-WORLD ANALOGY: A process is any employee at work; a daemon is one working silently in the back office; a service is that employee on an official, managed schedule.

Monitoring & Managing Processes

OBJECTIVE: See what's running, watch resource usage, and stop or manage it.

```
user@linux-server: ~  
  
$ ps aux  
USER  PID  CPU  MEM  COMMAND  
  
$ top  
(live resource dashboard)  
  
$ kill 4821  
(stops process ID 4821)  
  
$ systemctl status nginx  
(checks a service's health)
```

ps

List running processes

top / htop

Live, real-time resource monitor

kill

Stop a process by its PID

systemctl

Start, stop, restart & check services

Connecting to a Remote Server

OBJECTIVE: Understand how SSH lets you securely control a server from anywhere.

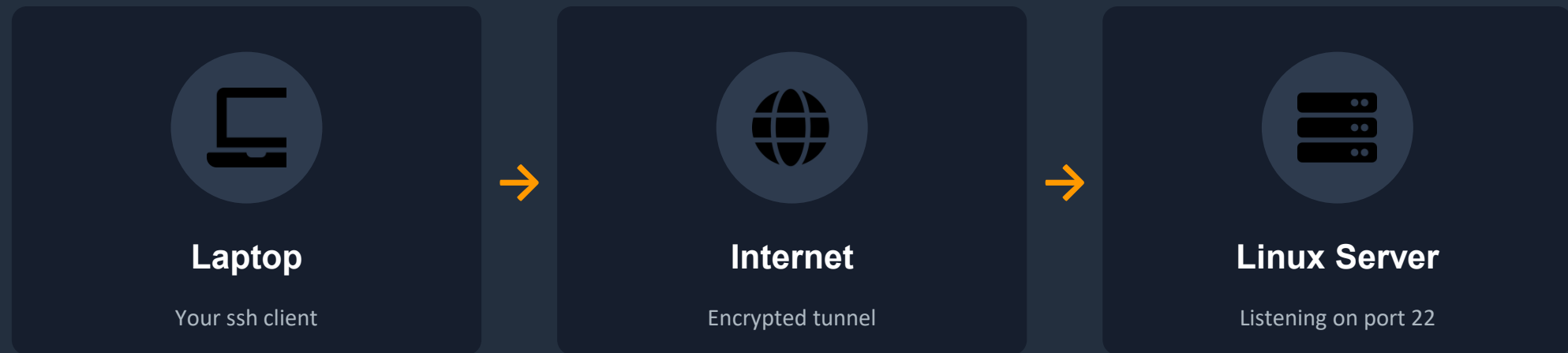
SSH (Secure Shell) lets you log into and control a remote Linux machine over the Internet, encrypted end-to-end.

- Encrypted connection
- Works over port 22
- Can use keys instead of passwords



REAL-WORLD ANALOGY: SSH is like a locked front door with a house key — only someone holding the exact matching key gets inside.

How an SSH Connection Travels



```
Terminal

$ ssh student@203.0.113.10
Welcome to Ubuntu 22.04 LTS
```

SSH Keys vs. Passwords

OBJECTIVE: Understand why key-based authentication is the professional standard.

```
user@linux-server: ~  
  
$ ssh-keygen -t rsa -b 4096  
  (generates a public/private key pair)  
  
$ ssh -i mykey.pem student@203.0.113.10  
  (connects using a private key file)
```

Public key

Lives on the server — safe to share, it's just a lock.

Private key

Stays on your laptop — never share it, it's the actual key.

Only a matching public/private pair unlocks the connection — no password to guess or steal.



INSTRUCTOR TIP: Never email or upload a .pem private key file anywhere — treat it exactly like a physical house key.

Put It All Together

A guided lab combining navigation, files, permissions, and processes.



Lab Steps — Part 1: Setup & Navigation

1

mkdir ~/lab && cd ~/lab

Create and enter a fresh lab folder

2

mkdir logs configs scripts

Create three subfolders

3

touch configs/app.conf

Create an empty config file

4

echo "debug=true" > configs/app.conf

Write a setting into the file

5

cat configs/app.conf

Confirm the content was saved

Lab Steps — Part 2: Permissions & Processes

6

touch scripts/deploy.sh

Create a placeholder script

7

chmod 755 scripts/deploy.sh

Make it executable

8

ls -l scripts/

Verify the new permissions

9

cp configs/app.conf configs/app.conf.bak

Back up the config file

10

ps aux | grep bash**INSTRUCTOR TIP:** Finished early? Try renaming deploy.sh with mv, then re-checking its permissions with ls -l.

Lab Activity — On Your Own

OBJECTIVE: Apply today's commands independently, without step-by-step guidance.

YOUR CHALLENGE

- 1 Create a project folder called website in your home directory
- 2 Inside it, create html/, css/, and scripts/ subfolders
- 3 Create an index.html file and add one line of text to it
- 4 Make a backup copy of index.html
- 5 Set scripts/ files to be executable (755) and html/ files to read-only for others (644)
- 6 Confirm every change using ls -l before moving on



15 minutes

Work solo or in pairs. Use the cheat sheet only if you get stuck.



Success Looks Like

A working folder structure, a readable index.html, a backup file, and correct permissions confirmed with ls -l.

Avoid These Early Pitfalls



Deleting Without Checking

Running `rm` without `ls`-ing first — no undo exists.



Using `sudo` Everywhere

Running every command as admin “just in case.”



Setting 777 Permissions

“Fixing” access errors by opening everything to everyone.



INSTRUCTOR TIP: Teach the habit early: `pwd` → `ls` → then act. Confirm before you commit.

Quiz — Questions 1–5

1

Which command shows your current directory?

2

Which command lists files, including hidden ones?

3

What does the kernel do?

4

What is the root of the Linux file system called?

5

Which command creates a new, empty file?

Quiz — Questions 6–10

6

What do the letters r, w, and x represent?

7

What does `chmod 755` grant to the Group category?

8

What command shows a live view of running processes?

9

What port does SSH use by default?

10

Why should you avoid `chmod 777` on real systems?

Quiz — Answer Key

1. pwd

2. ls -a

3. Manages hardware, memory & processes

4. / (root)

5. touch

6. rwx r-x r-x

7. Read & execute

8. top (or htop)

9. Port 22

10. It grants full access to every user — a major security risk

Homework



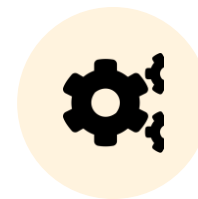
Practice Navigation

Spend 15 minutes creating, moving, and deleting test files/folders.



Set Custom Permissions

Create a script and set it to 755; a text file and set it to 644.



Explore Processes

Run top or htop and identify 3 processes currently running.

Cheat Sheet & Resources

pwd Show current directory

mkdir Make a folder

mv Move / rename

ps aux List processes

ls -la List all files, detailed

touch Make a file

rm -r Delete (recursive)

top Live resource monitor

cd ~ Go home

cp -r Copy (recursive for folders)

chmod 755 rwx r-x r-x

ssh user@ip Remote login

YOUTUBE RECOMMENDATIONS

- “Linux Crash Course” — NetworkChuck
- “Linux Command Line for Beginners” — freeCodeCamp
- “Linux Permissions Explained” — Techno Tim

Day 3: Networking Fundamentals

IP addresses, subnets, ports, and how AWS networking ties it all together.





Great Work Today!

You can now navigate, manage, and secure a Linux system with confidence.